

Java Interview Series · Part 9 · Q373–Q397

Modern Java 18 → 21

UTF-8 by default, virtual threads in depth, pinning, structured concurrency, scoped values, record patterns, pattern-matching switch, sequenced collections and the 17 → 21 upgrade.

25	20	9	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `MJ-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**
A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**
What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**
The next question the interviewer is likely to ask, with the answer.

25	5	20	9
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

🧭	01	Java 18 – 20	Q373–Q378	4
🧵	02	Virtual Threads in Depth (Java 21)	Q379–Q386	8
🌲	03	Structured Concurrency & Scoped Values	Q387–Q388	15
🧩	04	Pattern Matching & Data-Oriented Programming	Q389–Q392	17
2️⃣1️⃣	05	Java 21 APIs & the 17 → 21 Upgrade	Q393–Q397	21

© TechTribers. For personal use only. Do not redistribute.

All questions

🧭 Java 18 – 20

Q373 What changed when UTF-8 became the default charset in Java 18? 4

Q374 What is the Simple Web Server (`jwebserver`) in Java 18? 5

Q375 What are code snippets in Javadoc (Java 18)? 5

Q376 How was core reflection reimplemented in Java 18, and why does it matter? 6

Q377 How did Java 19 and 20 prepare the ground for Java 21? 7

Q378 What is the Vector API? 7

🧵 Virtual Threads in Depth (Java 21)

Q379 How do virtual threads work under the hood? 8

Q380 When should you use virtual threads, and when not? 9

Q381 What is thread pinning, and how was it fixed? 10

Q382 Can you use ThreadLocal with virtual threads? 11

Q383 How do you limit concurrency with virtual threads? 12

Q384 How do you enable virtual threads in Spring Boot, and what should you check? 13

Q385 How do you observe and debug virtual threads? 13

Q386 Virtual threads vs reactive programming (WebFlux, Reactor): which should you choose? 14

🌲 Structured Concurrency & Scoped Values

Q387 What is structured concurrency? 15

Q388 What are scoped values, and how do they compare with ThreadLocal? 16

🧩 Pattern Matching & Data-Oriented Programming

Q389 What are record patterns (Java 21)? 17

Q390 How does pattern matching for switch work: guards, null and dominance? 18

Q391 What is data-oriented programming in Java? 19

Q392 How do generics work with record patterns, and can `var` be inferred inside patterns? 20

2️⃣1️⃣ Java 21 APIs & the 17 → 21 Upgrade

Q393 What are sequenced collections (Java 21)? 21

Q394 Which small but useful APIs arrived in Java 21? 22

Q395 What is the Key Encapsulation Mechanism (KEM) API in Java 21? 23

Q396 Why does Java 21 warn about dynamically loaded agents? 24

Q397 Why upgrade from Java 17 to 21? What are the biggest gains? 24

© TechTribers. For personal use only. Do not redistribute.

Section 01

Java 18 – 20

UTF-8 everywhere, a built-in web server, executable doc snippets, and the preview releases that set up Java 21.

Questions 373–378 · 6 questions



Q373 MJ-034

• Intermediate

What changed when UTF-8 became the default charset in Java 18?

⚡ Quick answer

JEP 400 made UTF-8 the default for all standard Java APIs (`FileReader` , `FileWriter` , `String.getBytes()` , `Files` without a charset, etc.) on every OS. Before Java 18 the default followed the OS locale, often Windows-1252 on Windows, so the same code read files differently on different machines.

Migration impact: apps that relied on the old platform default to read legacy files (e.g. Windows-1252 CSVs) now decode them as UTF-8 and see garbled characters. Run with `-Dfile.encoding=COMPAT` temporarily, then fix the code to pass explicit charsets.

Java

```
System.out.println(Charset.defaultCharset());                    // UTF-8 on Java 18+
System.out.println(System.getProperty("file.encoding"));

Path p = Files.createTempFile("tt", ".txt");
try (var w = new FileWriter(p.toFile())) { w.write("₹499 · naïve café"); }   // no charset given
System.out.println(Files.readString(p).equals("₹499 · naïve café"));

// Console and native encodings are separate and may still follow the OS:
System.out.println(System.getProperty("native.encoding") != null);
```

▶ Output

```
UTF-8
UTF-8
true
true
```

⚠️ Interview trap

`System.out` still uses the console encoding (`stdout.encoding`), so `₹` can appear as `?` on an old Windows console even though files are written correctly.

© TechTribers. For personal use only. Do not redistribute.

Q374 MJ-035

• Basic

What is the Simple Web Server (`jwebserver`) in Java 18?

⚡ Quick answer

JEP 408 added `jwebserver`, a minimal static-file HTTP server for prototyping, testing and teaching, plus the `SimpleFileServer` API to start one from code. It serves files only (GET/HEAD), is HTTP/1.1, and isn't meant for production.

Java

```
Path root = Files.createTempDirectory("site");
Files.writeString(root.resolve("index.html"), "<h1>TechTribers</h1>");

var server = SimpleFileServer.createFileServer(new
InetSocketAddress(InetAddress.getLoopbackAddress(), 0),
        root, SimpleFileServer.OutputLevel.NONE);
server.start();
int port = server.getAddress().getPort();

HttpResponse<String> res = HttpClient.newHttpClient().send(
        HttpRequest.newBuilder(URI.create("http://127.0.0.1:" + port + "/index.html")).build(),
        HttpResponse.BodyHandlers.ofString());
System.out.println(res.statusCode() + " " + res.body());
server.stop(0);

// From a terminal:  jwebserver -p 8000 -d /path/to/site
```

▶ Output

```
200 <h1>TechTribers</h1>
```

💡 Pro tip

Great for serving a generated report, Javadoc or a test fixture to an integration test without adding a Jetty or Netty dependency.

Q375 MJ-036

• Intermediate

What are code snippets in Javadoc (Java 18)?

⚡ Quick answer

JEP 413 added the `{@snippet ...}` tag for code in Javadoc. Snippets can be inline or pulled from real source files by region, support highlighting and replacements, and (unlike `<pre>{@code}</pre>`) the external ones are compiled and tested like normal code.

© TechTribers. For personal use only. Do not redistribute.

```java
/**
 * Places an order and returns its id.
 *
 * {@snippet :
 * var service = new OrderService(repo);
 * String id = service.place(new Order("ORD-1", 499)); // @highlight substring="place"
 * }
 *
 * Example kept in a compiled, tested file:
 * {@snippet file="OrderExamples.java" region="refund"}
 */
public String place(Order order) { ... }
```

💡 **Pro tip**

Point external snippets at classes in your test source set: if the API changes, the build breaks instead of the docs silently going stale.

Q376 MJ-037 • Advanced

How was core reflection reimplemented in Java 18, and why does it matter?

⚡ **Quick answer**

JEP 416 rewrote `Method.invoke`, `Constructor.newInstance` and `Field.get/set` on top of method handles, replacing the old mix of native code and generated bytecode accessors. It simplified the JDK and made reflection performance more consistent.

What changed for applications:

- The first few reflective calls are no longer slow native calls followed by a sudden "inflation" into generated bytecode; performance is more predictable.
- Stack traces from reflective calls look different (method-handle frames instead of `GeneratedMethodAccessor` frames).
- Frameworks that heavily use reflection (Spring, Hibernate, Jackson) benefit without code changes.

⚠️ **Interview trap**

Old tuning flags such as `-Dsun.reflect.inflationThreshold` and `-Dsun.reflect.noInflation` no longer do anything.

© TechTribers. For personal use only. Do not redistribute.

Q377 MJ-038

• Intermediate

How did Java 19 and 20 prepare the ground for Java 21?

> ⚡ **Quick answer**
> Java 19 and 20 were almost entirely about **previews**: virtual threads, structured concurrency, record patterns, pattern matching for switch, the FFM API and scoped values. Java 21 then finalised virtual threads, record patterns and switch patterns, which is why it was such a significant LTS.

This shows how modern Java evolves: big features land as **incubator** (API in `jdk.incubator`) or **preview** (`--enable-preview`), collect feedback for a few releases, change, and then become final. Interview answers should mention status: "virtual threads were previewed in 19 and 20 and became final in 21".

Some features take many rounds: structured concurrency is still in preview in JDK 27 after a significant API redesign in JDK 25.

Preview pipeline: I = incubator, P = preview, ✓ = final

Virtual threads	19 P	20 P	21 ✓
Record patterns	19 P	20 P	21 ✓
Pattern switch	17 P	19 P	21 ✓
Structured concurrency	19 I	21 P	→ 27 P
Scoped values	20 I	21 P	25 ✓
FFM API	19 P	21 P	22 ✓

> ⚠️ **Interview trap**
> Code compiled with `--enable-preview` only runs on that exact JDK version. Shipping preview features in production locks you to one release.

Q378 MJ-039

• Advanced

What is the Vector API?

> ⚡ **Quick answer**
> The Vector API (`jdk.incubator.vector`, incubating since Java 16) expresses SIMD computations that the JIT compiles to vector instructions (AVX, NEON), giving several-fold speed-ups for numeric loops. It remains incubating until Project Valhalla's value classes arrive.

Use cases: machine-learning inference, image processing, cryptography, and analytics engines. The JIT already auto-vectorises simple loops; the Vector API is for cases it can't detect.

© TechTribers. For personal use only. Do not redistribute.

```java
// Run with: java --add-modules jdk.incubator.vector Main.java
static final VectorSpecies<Float> S = FloatVector.SPECIES_PREFERRED;

static void scaleAdd(float[] a, float[] b, float[] out, float k) {
    int i = 0;
    for (; i < S.loopBound(a.length); i += S.length()) {          // e.g. 8 floats per step on AVX2
        var va = FloatVector.fromArray(S, a, i);
        var vb = FloatVector.fromArray(S, b, i);
        va.mul(k).add(vb).intoArray(out, i);
    }
    for (; i < a.length; i++) out[i] = a[i] * k + b[i];           // scalar tail
}
```

💡 **Pro tip**

Because it's incubating, keep Vector API code behind an interface with a scalar fallback so the rest of the codebase doesn't depend on `jdk.incubator` modules.

Section 02

Virtual Threads in Depth (Java 21)

How virtual threads work, when they help, the pinning trap, and how to adopt them in real services.

Questions 379–386 · 8 questions



⚙️ **Q379** MJ-040 • Advanced

How do virtual threads work under the hood?

⚡ **Quick answer**

A virtual thread is a `Thread` scheduled by the JVM, not the OS. It runs ("mounts") on a carrier platform thread from a `ForkJoinPool`; when it blocks on I/O, a lock or `sleep`, its stack is saved to the heap as a continuation and the carrier is freed to run another virtual thread.

The JDK's blocking APIs (sockets, `Thread.sleep`, `BlockingQueue`, `ReentrantLock`, JDBC drivers using them) were reworked so they park the virtual thread instead of the OS thread. Your code stays plain, synchronous and readable.

```java
Thread vt = Thread.ofVirtual().name("order-", 0).start(() ->
        System.out.println(Thread.currentThread().getName() + " virtual=" +
```


© TechTribers. For personal use only. Do not redistribute.

```
Thread.currentThread().isVirtual()));
vt.join();

long start = System.nanoTime();
try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 100_000; i++) ex.submit(() -> { Thread.sleep(200); return null; });
}
long ms = (System.nanoTime() - start) / 1_000_000;
System.out.println("100,000 blocking tasks finished in under 5 s: " + (ms < 5_000));
```

▶ Output

```
order-0 virtual=true
100,000 blocking tasks finished in under 5 s: true
```

Virtual threads are mounted on a few carrier (platform) threads; blocking I/O unmounts them



Default carriers = number of CPU cores. Millions of virtual threads, a handful of OS threads.

💡 **Pro tip**

Virtual threads don't make code run faster; they let you run far **more** concurrent blocking operations. Throughput rises when the bottleneck was the number of threads.

⚖ Q380 MJ-041 • Intermediate

When should you use virtual threads, and when not?

⚡ **Quick answer**

Use them for high-concurrency, I/O-bound work: request handling, calls to databases, REST APIs and message brokers. Don't expect gains for CPU-bound work (use a pool sized to the cores) or when a downstream resource such as a DB connection pool is the real limit.

Rules of thumb:

- **One virtual thread per task**; never pool them.
- **Limit access to scarce resources** with a `Semaphore` or the connection pool itself, not by limiting threads.
- **CPU-heavy work** (image processing, encryption of large files, big computations) still belongs in a bounded platform-thread pool.
- **Existing thread-per-request code** (Spring MVC, JDBC) benefits most, with almost no code changes.

© TechTribers. For personal use only. Do not redistribute.

	Platform thread	Virtual thread
Managed by	OS kernel	JVM scheduler
Creation cost	~1 ms, ~1 MB stack reserved	microseconds, small heap-allocated stack
How many	thousands	millions
Blocking I/O	holds an OS thread	unmounts, carrier keeps working
Pool them?	yes (expensive)	never: one per task
Good for	CPU-bound work	I/O-bound request handling

⚠️ **Interview trap**

Swapping 200 platform threads for 10,000 virtual threads in front of a 20-connection Hikari pool just moves the queue: requests now wait on `getConnection()` instead of on threads.

📌 **Q381** `MJ-042` • Advanced

What is thread pinning, and how was it fixed?

⚡ **Quick answer**

A virtual thread is **pinned** when it can't unmount while blocked, so its carrier thread blocks too. On JDK 21–23 this happened when blocking inside a `synchronized` block or method; JDK 24 (JEP 491) removed that case. Blocking inside native or foreign-function calls still pins.

Java

```
class LegacyCache {
    private final Map<String, String> data = new HashMap<>();
    // JDK 21-23: blocking I/O inside synchronized pins the carrier
    synchronized String load(String key) throws Exception {
        return data.computeIfAbsent(key, k -> fetchFromNetwork(k));   // slow call under the monitor
    }
    // Portable fix for older JDKs: ReentrantLock lets the virtual thread unmount
    private final ReentrantLock lock = new ReentrantLock();
    String loadSafe(String key) {
        lock.lock();
        try { return data.computeIfAbsent(key, k -> fetchFromNetwork(k)); }
        finally { lock.unlock(); }
    }
    static String fetchFromNetwork(String k) { return "v-" + k; }
}
System.out.println(new LegacyCache().loadSafe("a"));
```

▶ **Output**

```
v-a
```


© TechTribers. For personal use only. Do not redistribute.

Pinning: when a virtual thread can't unmount while it blocks, its carrier is blocked too

`synchronized + blocking I/O`	pinned on JDK 21–23	fixed in JDK 24 (JEP 491)
`native method / foreign call`	pinned	still pins: keep native calls short
`ReentrantLock + blocking I/O`	unmounts normally	fine on every version

Find it: JFR event jdk.VirtualThreadPinned (and -Djdk.tracePinnedThreads on JDK 21–23).

💡 **Pro tip**

Detect pinning with the JFR event `jdk.VirtualThreadPinned` (emitted when a pinned thread blocks too long). On JDK 21–23 you can also run with `-Djdk.tracePinnedThreads=full`.

🧳 Q382 MJ-043 • Advanced

Can you use ThreadLocal with virtual threads?

⚡ **Quick answer**

Yes, it works and is per virtual thread, but it's costly at scale: millions of virtual threads each holding their own copies use a lot of memory, and caching expensive objects in a `ThreadLocal` no longer helps because virtual threads aren't reused. Prefer passing context explicitly or using scoped values.

Java

```java
ThreadLocal<String> user = new ThreadLocal<>();
try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
    for (String u : List.of("asha", "ravi")) {
        ex.submit(() -> {
            user.set(u);
            try { System.out.println(Thread.currentThread().isVirtual() + " -> " + user.get()); }
            finally { user.remove(); }
        });
    }
}
```

▶ **Output**

```
true -> asha
true -> ravi
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

A common anti-pattern is a `ThreadLocal<SimpleDateFormat>` or `ThreadLocal<ObjectMapper>` cache. With virtual threads it creates a new instance per task, so it becomes pure overhead. Use thread-safe objects (`DateTimeFormatter` , a shared `ObjectMapper`).

🚦 **Q383** `MJ-044` • Intermediate

How do you limit concurrency with virtual threads?

⚡ **Quick answer**

Don't size a pool of virtual threads. Create one per task and protect the limited resource with a `Semaphore` (or the resource's own pool), so millions of tasks can wait cheaply while only N touch the downstream system at once.

Java

```
Semaphore partnerApiSlots = new Semaphore(10);          // partner allows 10 concurrent calls
AtomicInteger inFlight = new AtomicInteger(), peak = new AtomicInteger();

try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 1_000; i++) {
        ex.submit(() -> {
            partnerApiSlots.acquire();
            try {
                peak.accumulateAndGet(inFlight.incrementAndGet(), Math::max);
                Thread.sleep(5);                            // the remote call
            } finally {
                inFlight.decrementAndGet();
                partnerApiSlots.release();
            }
            return null;
        });
    }
}
System.out.println("max concurrent partner calls = " + peak.get());
```

▶ **Output**

```
max concurrent partner calls = 10
```

💡 **Pro tip**

Rate limits (requests per second) are different from concurrency limits. Use a rate limiter (Resilience4j, Bucket4j) for the former; a semaphore only bounds how many calls run at the same time.

© TechTribers. For personal use only. Do not redistribute.

Q384 MJ-045

• Intermediate

How do you enable virtual threads in Spring Boot, and what should you check?

⚡ **Quick answer**

Spring Boot 3.2+ on Java 21: set `spring.threads.virtual.enabled=true`. Tomcat/Jetty then handle each request on a virtual thread, and `@Async`, scheduling and many Spring executors use virtual threads too. Then re-check connection pools, `synchronized` hot spots (on JDK 21–23) and ThreadLocal-heavy libraries.

Checklist before turning it on in production:

- Load-test: watch DB pool wait time, downstream latency and error rates, not just throughput.
- On JDK 21–23, look for pinning from `synchronized` around I/O in your code and libraries (older JDBC drivers, some caches); JDK 24+ removes most of it.
- Make sure HTTP clients have timeouts; with more concurrency, a hung dependency now ties up thousands of requests instead of 200.

Terminal

```
# application.yml
spring:
  threads:
    virtual:
      enabled: true          # Spring Boot 3.2+, Java 21+
  datasource:
    hikari:
      maximum-pool-size: 30  # the real concurrency limit for DB work
server:
  tomcat:
    threads:
      max: 200               # ignored for request handling once virtual threads are enabled
```

💡 **Pro tip**

Virtual threads let Spring MVC with JDBC reach throughput that previously required WebFlux and R2DBC, while keeping simple imperative code and normal stack traces.

Q385 MJ-046

• Advanced

How do you observe and debug virtual threads?

⚡ **Quick answer**

Classic `jstack` dumps show only platform threads. Use `jcmd <pid> Thread.dump_to_file -format=json` for a full dump including virtual threads, JFR events (`jdk.VirtualThreadStart`, `jdk.VirtualThreadEnd`, `jdk.VirtualThreadPinned`, `jdk.VirtualThreadSubmitFailed`), and debuggers, which support them natively.

© TechTribers. For personal use only. Do not redistribute.

Terminal

```
# Full dump including virtual threads, grouped by executor / structured scope
jcmd 4321 Thread.dump_to_file -format=json /tmp/threads.json

# Record virtual-thread events
jcmd 4321 JFR.start name=vt duration=60s filename=/tmp/vt.jfr
jfr print --events jdk.VirtualThreadPinned /tmp/vt.jfr

# Tune the scheduler if needed (defaults = number of cores)
java -Djdk.virtualThreadScheduler.parallelism=8 -Djdk.virtualThreadScheduler.maxPoolSize=256 -jar
app.jar
```

⚠️ **Interview trap**

Thread-count dashboards (e.g. `jvm.threads.live`) count platform threads only; they won't show a backlog of virtual threads. Watch request latency, queue times and pool wait metrics instead.

🔄 **Q386** MJ-047 • Advanced

Virtual threads vs reactive programming (WebFlux, Reactor): which should you choose?

⚡ **Quick answer**

Both solve "too many threads blocked on I/O". Reactive does it with non-blocking callbacks and operators; virtual threads keep ordinary blocking code and make blocking cheap. For new I/O-bound services, virtual threads are usually simpler; reactive still wins for streaming, back-pressure-heavy pipelines and existing reactive stacks.

	Virtual threads	Reactive (WebFlux/Reactor)
Code style	Plain imperative, `try/catch`, loops	Operator chains (`flatMap`, `zip`), callbacks
Debugging	Normal stack traces and debuggers	Hard: async stack traces, assembly tracing
Libraries	Any blocking library (JDBC, JPA)	Needs non-blocking drivers (R2DBC, reactive clients)
Back-pressure	Manual (semaphores, bounded queues)	Built in
Streaming	Possible but manual	Natural fit (SSE, WebSockets, Kafka streams)

💡 **Pro tip**

A strong interview answer avoids dogma: "For a CRUD service on Postgres, Spring MVC + virtual threads; for a streaming gateway with back-pressure, WebFlux."

© TechTribers. For personal use only. Do not redistribute.

Section 03

Structured Concurrency & Scoped Values



Project Loom's other two pieces: treating related tasks as one unit, and sharing context safely with millions of threads.

Questions 387–388 · 2 questions

Q387 MJ-048

• Advanced

What is structured concurrency?

⚡ Quick answer

Structured concurrency treats a group of concurrent subtasks as one unit of work with a clear scope: subtasks are forked inside a `StructuredTaskScope`, the parent waits for them, and when one fails (or the policy is met) the others are cancelled. It's been in preview since Java 21 and was redesigned in Java 25.

Problems it solves compared with `ExecutorService` + `Future`:

- **Leaks**: with plain futures, if `fetchUser` fails, `fetchOrder` keeps running and nobody cancels it.
- **Cancellation**: interrupting the parent cancels all children automatically.
- **Observability**: thread dumps show the parent/child task tree.

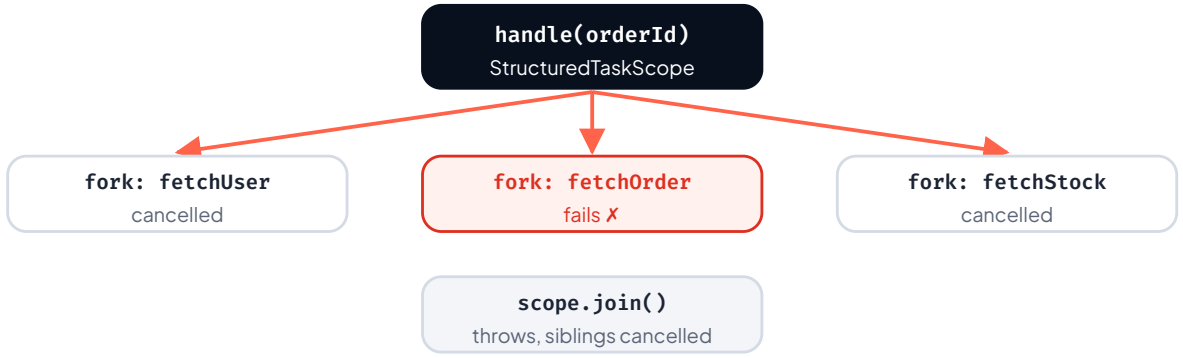
Java

```
// Java 25+ preview API: compile and run with --enable-preview
record Page(User user, Order order) { }

Page load(long userId, long orderId) throws InterruptedException {
    try (var scope = StructuredTaskScope.open()) {              // default: fail fast on first
error
        Subtask<User> user   = scope.fork(() -> userService.find(userId));
        Subtask<Order> order = scope.fork(() -> orderService.find(orderId));
        scope.join();                                           // waits; throws if any subtask
failed
        return new Page(user.get(), order.get());
    }   // leaving the scope guarantees no subtask is still running
}

// Other policies via Joiner, e.g. first successful result wins:
// try (var scope = StructuredTaskScope.open(Joiner.<String>anySuccessfulResultOrThrow())) { ... }
```


© TechTribers. For personal use only. Do not redistribute.



Child tasks can't outlive the scope: one failure cancels the rest, and thread dumps show the parent/child tree.

⚠️ **Interview trap**

The API changed between previews (Java 21's `ShutdownOnFailure` became `open()` with `Joiner` policies in Java 25). Interview answers should describe the concept and mention that it's still a preview feature in JDK 27.

🎯 **Q388** MJ-049 • Advanced

What are scoped values, and how do they compare with ThreadLocal?

⚡ **Quick answer**

A `ScopedValue` (final in Java 25) shares an immutable value with everything a thread calls within a bounded scope, and is automatically unbound when the scope ends. It's cheaper and safer than `ThreadLocal`, especially with millions of virtual threads and structured subtasks that inherit it.

Use cases: request context (tenant, user, trace ID), security principal, transaction context in frameworks. To change a value for a sub-call, **rebind** it with a nested `where(...).run(...)` instead of mutating it.

Java

```java
// Java 25+
static final ScopedValue<String> TENANT = ScopedValue.newInstance();

void handle(Request req) {
    ScopedValue.where(TENANT, req.tenantId())
               .run(() -> orderService.process(req));     // TENANT bound only inside run()
}

class OrderRepository {
    List<Order> findAll() {
        String tenant = TENANT.get();                     // read anywhere below in the call chain
        return jdbc.query("SELECT * FROM orders WHERE tenant_id = ?", tenant);
    }
}
// Outside the scope: TENANT.isBound() == false, TENANT.get() throws NoSuchElementException
```


© TechTribers. For personal use only. Do not redistribute.

ThreadLocal
- mutable: set() anywhere
- lives as long as the thread
- must remove() in finally
- copied into child threads
- (InheritableThreadLocal): costly

ScopedValue (final in Java 25)
- immutable for the scope
- bound only inside where(...).run()
- unbound automatically on exit
- shared cheaply with forked
- subtasks in structured scopes

💡 **Pro tip**

Scoped values were in incubator/preview from Java 20 to 24 and are final in Java 25, so production code on 25+ can use them without flags.

Section 04

Pattern Matching & Data-Oriented Programming

Record patterns, switch patterns with guards and null handling, and why they change how Java models data.

Questions 389–392 · 4 questions

🔍 **Q389** MJ-050 • Intermediate

What are record patterns (Java 21)?

⚡ **Quick answer**

Record patterns deconstruct a record into its components directly in `instanceof` or `switch`, and they nest: `if (obj instanceof Order(Customer(var name, var tier), var lines))` checks the types and binds `name`, `tier` and `lines` in one step.

Java

```
enum Tier { SILVER, GOLD }
record Customer(String name, Tier tier) { }
record Line(String sku, int qty) { }
record Order(Customer customer, List<Line> lines) { }

static String describe(Object o) {
    if (o instanceof Order(Customer(var name, var tier), var lines) && tier == Tier.GOLD)
        return "VIP order from " + name + " with " + lines.size() + " lines";
```


© TechTribers. For personal use only. Do not redistribute.

```
    if (o instanceof Order(Customer c, var lines))
        return "order from " + c.name();
    return "unknown";
}
var gold = new Order(new Customer("Asha", Tier.GOLD), List.of(new Line("BOOK", 2), new Line("PEN",
5)));
var silver = new Order(new Customer("Ravi", Tier.SILVER), List.of());
System.out.println(describe(gold) + " | " + describe(silver) + " | " + describe("x"));
```

▶ Output

```
VIP order from Asha with 2 lines | order from Ravi | unknown
```



⚠️ **Interview trap**

A record pattern never matches `null`: `o instanceof Order(var c, var l)` is `false` for a null `o`, and a nested pattern doesn't match a record whose component is `null` unless that component's pattern is a plain type or `var`.

Q390 MJ-051 • Advanced

How does pattern matching for switch work: guards, null and dominance?

⚡ **Quick answer**

Since Java 21, `case` labels can be type patterns and record patterns, refined with `when` guards. `case null` handles nulls explicitly (otherwise switch throws NPE), cases are checked top-down, and the compiler rejects a case dominated by an earlier one.

Java

```
sealed interface Shape permits Circle, Square, Rect { }
record Circle(double r) implements Shape { }
record Square(double side) implements Shape { }
record Rect(double w, double h) implements Shape { }

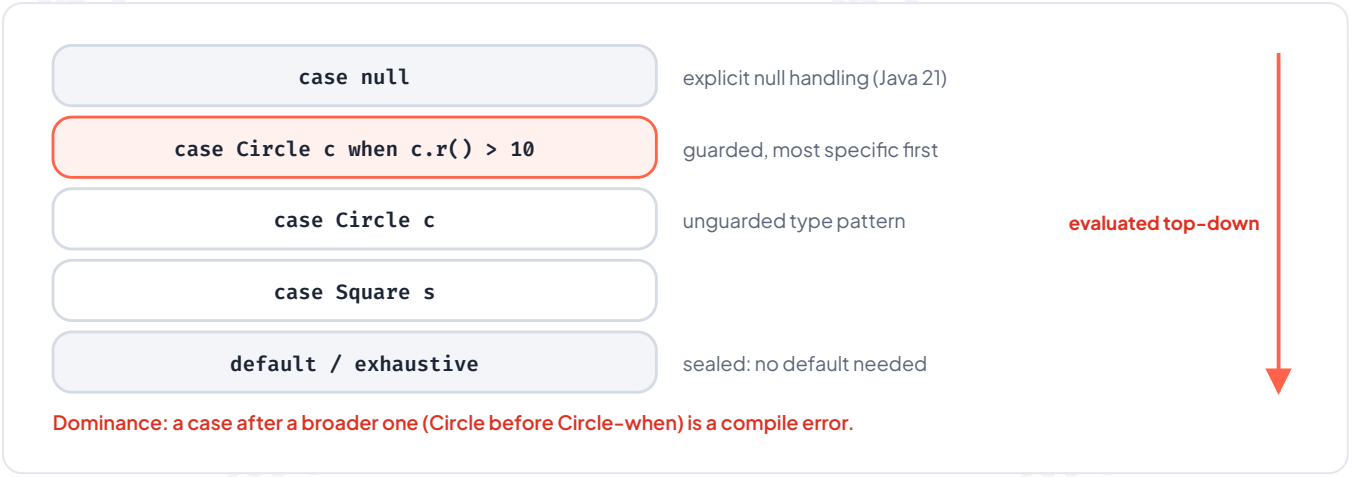
static String classify(Shape s) {
    return switch (s) {
        case null -> "no shape";
```


© TechTribers. For personal use only. Do not redistribute.

```
        case Circle c when c.r() > 10 -> "large circle";
        case Circle c -> "circle r=" + c.r();
        case Rect(double w, double h) when w == h -> "square-ish rect " + w;
        case Rect r -> "rect " + r.w() + "x" + r.h();
        case Square sq -> "square " + sq.side();
    };                                                  // exhaustive over the sealed type: no
default
}
for (Shape s : new Shape[]{new Circle(12), new Circle(3), new Rect(4, 4), new Rect(2, 5), new
Square(1), null})
    System.out.println(classify(s));
```

▶ Output

```
large circle
circle r=3.0
square-ish rect 4.0
rect 2.0x5.0
square 1.0
no shape
```



⚠️ **Interview trap**

Swapping the two `Circle` cases is a compile error: `case Circle c` dominates `case Circle c when ...`, so the guarded case could never run.

Q391 MJ-052 • Advanced

What is data-oriented programming in Java?

⚡ **Quick answer**

Model data as immutable **records**, model alternatives as **sealed** hierarchies, and write operations as **pattern-matching switches** over them. The compiler then checks that every case is handled, which suits APIs, events and domain results better than deep class hierarchies with overridden methods.

It's the functional-programming idea of algebraic data types, now native in Java. Object-oriented polymorphism is still right when behaviour belongs to the type and new types are added often; data-oriented style is right when

© TechTribers. For personal use only. Do not redistribute.

new **operations** are added often.

```java
sealed interface PaymentEvent permits Authorized, Captured, Refunded, Failed { }
record Authorized(String id, long paise) implements PaymentEvent { }
record Captured(String id, long paise) implements PaymentEvent { }
record Refunded(String id, long paise, String reason) implements PaymentEvent { }
record Failed(String id, String code) implements PaymentEvent { }

static long balanceChange(PaymentEvent e) {
    return switch (e) {
        case Authorized a -> 0;
        case Captured(var id, var p) -> p;
        case Refunded(var id, var p, var reason) -> -p;
        case Failed f -> 0;
    };                                  // add a new event type -> compile error here until handled
}
List<PaymentEvent> events = List.of(new Authorized("P1", 49_900), new Captured("P1", 49_900),
        new Refunded("P1", 9_900, "damaged"), new Failed("P2", "INSUFFICIENT_FUNDS"));
System.out.println(events.stream().mapToLong(e -> balanceChange(e)).sum());
```

▶ Output

```
40000
```

💡 **Pro tip**

This pattern replaces the Visitor pattern in most code: no `accept` / `visit` boilerplate, and the exhaustiveness check gives the same safety.

Q392 MJ-053 • Advanced

How do generics work with record patterns, and can `var` be inferred inside patterns?

⚡ **Quick answer**

Record patterns infer type arguments: `case Box(var v)` against a `Box<String>` binds `v` as `String`, and you can write `Box(String s)` without repeating `<String>`. `var` in a nested pattern takes the component's declared type.

```java
record Box<T>(T value) { }
record Pair<A, B>(A first, B second) { }

static String show(Object o) {
    return switch (o) {
        case Box(String s) -> "box of text: " + s.toUpperCase();
        case Box(Integer i) when i > 100 -> "box of big number: " + i;
        case Box(var v) -> "box of " + (v == null ? "null" : v.getClass().getSimpleName());
```


© TechTribers. For personal use only. Do not redistribute.

```
        case Pair(Box(var x), Box(var y)) -> "pair of boxes: " + x + ", " + y;
        default -> "something else";
    };
}
System.out.println(show(new Box<>("java")));
System.out.println(show(new Box<>(250)));
System.out.println(show(new Box<>(3.5)));
System.out.println(show(new Pair<>(new Box<>(1), new Box<>("b"))));
```

▶ Output

```
box of text: JAVA
box of big number: 250
box of Double
pair of boxes: 1, b
```

Section 05

Java 21 APIs & the 17 → 21 Upgrade

Sequenced collections, handy new methods, post-quantum-ready crypto plumbing, agent warnings, and why 21 became the default LTS.

Questions 393–397 · 5 questions



Q393 MJ-054 • Intermediate

What are sequenced collections (Java 21)?

⚡ Quick answer

JEP 431 added `SequencedCollection`, `SequencedSet` and `SequencedMap`: a common interface for collections with a defined encounter order, with `getFirst`, `getLast`, `addFirst`, `addLast`, `removeFirst`, `removeLast` and a `reversed()` view. `List`, `Deque`, `LinkedHashSet`, `TreeSet`, `LinkedHashMap` and `TreeMap` implement them.

Java

```
List<String> list = new ArrayList<>(List.of("b", "c"));
list.addFirst("a"); list.addLast("d");
System.out.println(list + " first=" + list.getFirst() + " last=" + list.getLast() + " reversed=" +
list.reversed());

LinkedHashSet<Integer> set = new LinkedHashSet<>(List.of(3, 1, 2));
set.addFirst(9);                                    // moves/inserts at the front
System.out.println(set + " " + set.reversed());

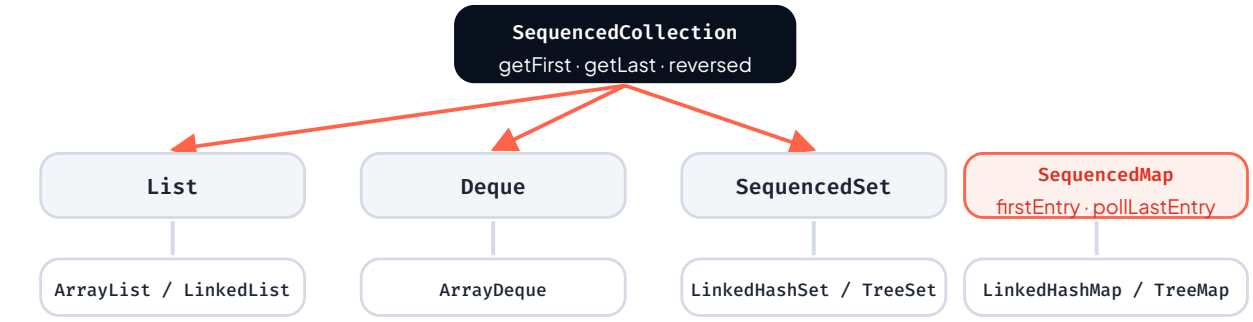
LinkedHashMap<String, Integer> map = new LinkedHashMap<>();
map.put("x", 1); map.put("y", 2); map.putFirst("w", 0);
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(map.firstEntry() + " " + map.lastEntry() + " " + map.pollLastEntry() + " -> " +
map);
```

▶ Output

```
[a, b, c, d] first=a last=d reversed=[d, c, b, a]
[9, 3, 1, 2] [2, 1, 3, 9]
w=0 y=2 y=2 -> {w=0, x=1}
```



One API for 'first and last' everywhere. Before Java 21: list.get(list.size()-1), deque.peekLast(), treeSet.last()...

⚠️ **Interview trap**

`reversed()` returns a **view**, not a copy: changes through it affect the original. Also, unmodifiable lists (`List.of`) throw on `addFirst` / `removeLast` like any other mutator.

Q394 MJ-055 • Intermediate

Which small but useful APIs arrived in Java 21?

⚡ **Quick answer**

Highlights: `StringBuilder.repeat` , `String.indexOf(ch, from, to)` , `String.splitWithDelimiters` , `Character.isEmoji` and related emoji methods, `Math.clamp` , and `HttpClient` implementing `AutoCloseable` .

Java

```
StringBuilder bar = new StringBuilder().repeat("▓", 5).repeat("░", 3);
System.out.println(bar);

System.out.println(Math.clamp(150, 0, 100) + " " + Math.clamp(-5, 0, 100) + " " + Math.clamp(42, 0,
100));

String s = "a,b;c";
System.out.println(Arrays.toString(s.splitWithDelimiters("[,;]", 0)));

int rocket = "🚀".codePointAt(0);
System.out.println(Character.isEmoji(rocket) + " " + Character.isEmoji('A'));
System.out.println("hello world".indexOf('o', 5, 11));
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output



```
100 0 42
[a, ,, b, ;, c]
true false
7
```

💡 **Pro tip**

`Math.clamp` removes a whole class of hand-written `Math.max(min, Math.min(max, v))` bugs where the arguments get swapped.

🔐 **Q395** MJ-056 • Advanced

What is the Key Encapsulation Mechanism (KEM) API in Java 21?

⚡ **Quick answer**

JEP 452 added `javax.crypto.KEM`, a standard API for key encapsulation: the sender uses the receiver's public key to generate a fresh shared secret plus an encapsulation to send; the receiver decapsulates it to get the same secret. It's the building block for hybrid and post-quantum TLS (ML-KEM arrived in Java 24).

Why it matters now: "harvest now, decrypt later" attacks motivate post-quantum key exchange. Java 24 added ML-KEM and ML-DSA (FIPS 203/204), and JDK 27 adds post-quantum hybrid key exchange for TLS 1.3, all built on this API style.

Java

```java
KeyPairGenerator g = KeyPairGenerator.getInstance("X25519");
KeyPair receiver = g.generateKeyPair();

KEM kem = KEM.getInstance("DHKEM");
KEM.Encapsulator enc = kem.newEncapsulator(receiver.getPublic());
KEM.Encapsulated sent = enc.encapsulate();                    // sender: secret + bytes to send

KEM.Decapsulator dec = kem.newDecapsulator(receiver.getPrivate());
SecretKey recovered = dec.decapsulate(sent.encapsulation());    // receiver

System.out.println("shared secrets match: " + Arrays.equals(sent.key().getEncoded(),
recovered.getEncoded()));
System.out.println("secret length: " + recovered.getEncoded().length + " bytes");
```

▶ Output

```
shared secrets match: true
secret length: 32 bytes
```


© TechTribers. For personal use only. Do not redistribute.

Q396 MJ-057

• Advanced

Why does Java 21 warn about dynamically loaded agents?

> ⚡ **Quick answer**
> JEP 451 makes the JVM print a warning when a Java agent is loaded into a **running** JVM (via the Attach API), and a future release will disallow it by default. Agents loaded at start-up with `-javaagent` are unaffected. The goal is integrity: code shouldn't be able to silently rewrite a running application.

Who is affected: Mockito's inline mock maker and Byte Buddy (they self-attach in tests), some profilers and diagnostic tools attached on demand. Mockito 5 documents adding Mockito as a `-javaagent` in the test JVM.

Terminal

```
# Fine: agent declared at launch (APM tools, coverage)
java -javaagent:/opt/otel/opentelemetry-javaagent.jar -jar app.jar

# Warns on 21+ (and will need opt-in): attaching to a running JVM
#   WARNING: A Java agent has been loaded dynamically (...)
# Explicit opt-in silences it:
java -XX:+EnableDynamicAgentLoading -jar app.jar
```

> 💡 **Pro tip**
> This is part of a wider JDK theme called "integrity by default", alongside strong encapsulation of internals and restrictions on JNI and `Unsafe` memory access.

Q397 MJ-058

• Intermediate

Why upgrade from Java 17 to 21? What are the biggest gains?

> ⚡ **Quick answer**
> Java 21 is the LTS that finalised virtual threads, record patterns and pattern matching for switch, and added sequenced collections and generational ZGC. For typical Spring Boot services it means higher throughput with simple blocking code, cleaner domain modelling and better tail latency, usually with a low-risk upgrade from 17.

Release	Highlights
18	UTF-8 by default, `jwebserver`, Javadoc snippets, reflection on method handles
19	Previews: virtual threads, record patterns, FFM; structured concurrency incubator
20	Further previews; scoped values incubator
21 (LTS)	Virtual threads, record patterns, switch patterns, sequenced collections, generational ZGC, KEM API, dynamic-agent warnings

© TechTribers. For personal use only. Do not redistribute.

Upgrade checklist from 17: bump the build toolchain and bytecode libraries, run the test suite with `-XX:+EnableDynamicAgentLoading` only if needed for mocks, consider generational ZGC for latency-critical services, then enable virtual threads behind a feature flag and load-test.

> 💡 **Pro tip**
> Spring Boot 3.2+ with Java 21 is the common production baseline in 2026; Java 25 LTS is the next step for compact object headers, the AOT cache and final scoped values.

Series progress

397 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 10 starts at Q398.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47
Part 3	Q122–Q170	49
Part 4	Q171–Q204	34
Part 5	Q205–Q254	50
Part 6	Q255–Q304	50
Part 7	Q305–Q339	35
Part 8	Q340–Q372	33
Part 9	Q373–Q397	25

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

© TechTribers. For personal use only. Do not redistribute.